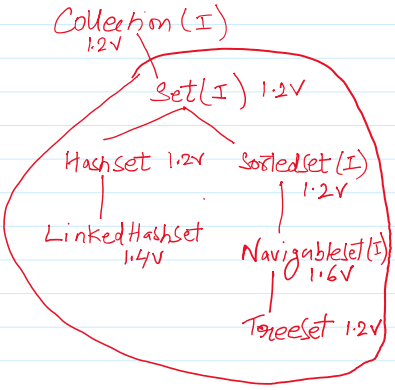
Set Interface

30 March 2025 10:26

Set is a child interface of Collection interface. If we want to represent a group of individual object as a single entity where duplicates are not allowed and insertion order is not preserved.

Set interface doesn't contain any new method and we have to use only Collection interface methods.



Official Documentation:

```
public interface Set<E>
extends Collection<E>
```

A collection that contains no duplicate elements. More formally, sets contain no pair of elements `e1` and `e2` such that `e1.equals(e2)`, and at most one null element. As implied by its name, this interface models the mathematical *set* abstraction.

The `Set` interface places additional stipulations, beyond those inherited from the `Collection` interface, on the contracts of all constructors and on the contracts of the `add`, `equals` and `hashCode` methods. Declarations for other inherited methods are also included here for convenience. (The specifications accompanying these declarations have been tailored to the `Set` interface, but they do not contain any additional stipulations.)

The additional stipulation on constructors is, not surprisingly, that all constructors must create a set that contains no duplicate elements (as defined above).

Note: Great care must be exercised if mutable objects are used as set elements. The behavior of a set is not specified if the value of an object is changed in a manner that affects `equals` comparisons while the object is an element in the set. A special case of this prohibition is that it is not permissible for a set to contain itself as an element.

Some set implementations have restrictions on the elements that they may contain. For example, some implementations prohibit null elements, and some have restrictions on the types of their elements. Attempting to add an ineligible element throws an unchecked exception, typically `NullPointerException` or `ClassCastException`. Attempting to query the presence of an ineligible element may throw an exception, or it may simply return false; some implementations will exhibit the former behavior and some will exhibit the latter. More generally, attempting an operation on an ineligible element whose completion would not result in the insertion of an ineligible element into the set may throw an exception or it may succeed, at the option of the implementation. Such exceptions are marked as "optional" in the specification for this interface.

This interface is a member of the Java Collections Framework.

Since:
`1.2`

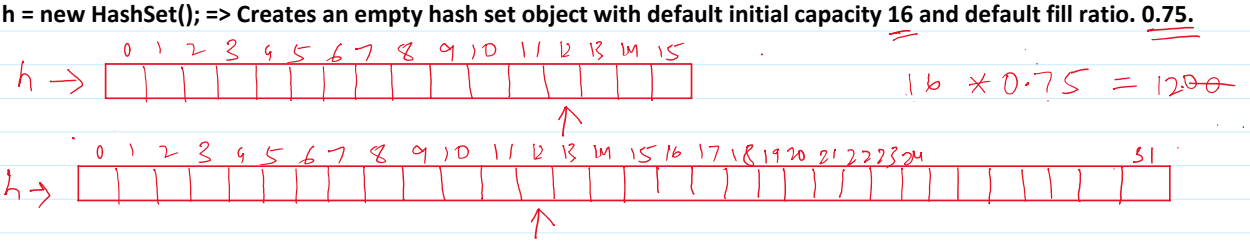
HashSet

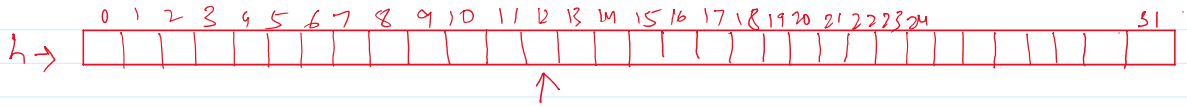
- The Underline data structure is HashTable.
- Duplicate objects are not allowed.
- Insertion order is not preserved.
- And it is based on hash code of object.
- Null insertion is possible only once.
- Heterogeneous objects are allowed.
- Implements Serializable and Clonable interfaces, but not RandomAccess interface.
- HashSet is the best choice if our frequent operation is search operation.

Note: In hash set duplicates are not allowed. If we are trying to insert duplicates, then we would not get any compile time or runtime error. The add() method simply returns false.

Constructors

1. **HashSet h = new HashSet(); => Creates an empty hash set object with default initial capacity 16 and default fill ratio. 0.75.**





2. **HashSet h = new HashSet(int initialCapacity); => Creates an empty hashtag object with a specified initial capacity and default fill ratio. 0.75.**
3. **HashSet h = new HashSet(int initialCapacity, float fillRatio);** $0.0 < fill_ratio < 1.0$
4. **HashSet h = new HashSet(Collection c); => Creates an equivalent hash set for a given collection. This constructor is meant for inter conversion between collection object.**

Note: Fill ratio or load factor: After filling how much ratio a new has set object will be created. This ratio is called fill ratio, or load factor. For example, fill ratio, 0.75 means after filling 75% a new hash set object will be created.

Official Documentation:
public class HashSet<E>
extends AbstractSet<E>
implements Set<E>, Cloneable, Serializable
This class implements the Set interface, backed by a hash table (actually a HashMap instance). It makes no guarantees as to the iteration order of the set; in particular, it does not guarantee that the order will remain constant over time. This class permits the null element.

*****This class offers constant time performance for the basic operations (add, remove, contains and size), assuming the hash function disperses the elements properly among the buckets. Iterating over this set requires time proportional to the sum of the HashSet instance's size (the number of elements) plus the "capacity" of the backing HashMap instance (the number of buckets). Thus, it's very important not to set the initial capacity too high (or the load factor too low) if iteration performance is important.**

Note that this implementation is not synchronized. If multiple threads access a hash set concurrently, and at least one of the threads modifies the set, it *must* be synchronized externally. This is typically accomplished by synchronizing on some object that naturally encapsulates the set. If no such object exists, the set should be "wrapped" using the Collections.synchronizedSet method. This is best done at creation time, to prevent accidental unsynchronized access to the set:
Set s = Collections.synchronizedSet(new HashSet(...));
The iterators returned by this class's iterator method are *fail-fast*: if the set is modified at any time after the iterator is created, in any way except through the iterator's own remove method, the Iterator throws a ConcurrentModificationException. Thus, in the face of concurrent modification, the iterator fails quickly and cleanly, rather than risking arbitrary, non-deterministic behavior at an undetermined time in the future.
Note that the fail-fast behavior of an iterator cannot be guaranteed as it is, generally speaking, impossible to make any hard guarantees in the presence of unsynchronized concurrent modification. Fail-fast iterators throw ConcurrentModificationException on a best-effort basis. Therefore, it would be wrong to write a program that depended on this exception for its correctness: *the fail-fast behavior of iterators should be used only to detect bugs.*
This class is a member of the Java Collections Framework.
Since:
1.2

From <https://docs.oracle.com/javase/8/docs/api/java/util/HashSet.html>

```
import java.util.HashSet;
public class MyHashSet {
    public static void main(String[] args) {
        HashSet h = new HashSet();
        System.out.println("Hash Set: ");
        h.add("B");
        h.add("C");
        h.add("D");
        h.add("Z");
        h.add(10);
        h.add("M");
        h.add("E");
        System.out.println(h.add("Z"));
        System.out.println(h);
```

```
            h.add("M");
            h.add("E");
            System.out.println(h.add("Z"));
            System.out.println(h);
        }
    }
```

Output:

```
Hash Set:
false
[B, C, D, E, Z, 10, M]
```

insertion order is not preserved.

LinkedHashSet

HashSet	**LinkedHashSet**
Underlying data structure is Hash table.	Underlying data structure is LinkedList + Hash table.
Insertion order is not preserved.	Insertion order is preserved.
Introduced in version 1.2	Introduced in version 1.4.

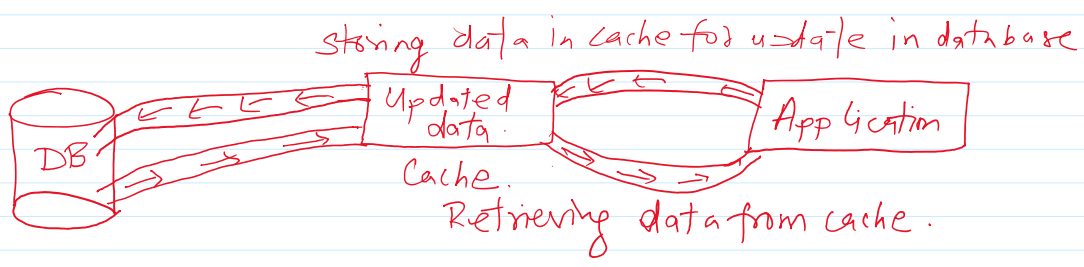
LinkedHashSet is a child class of HashSet. It is exactly same as HashSet(including Constructors and Methods) Except the following difference:
The demo program is written for HashSet, if we replace HashSet with LinkedHashSet then object will preserve insertion order.

```
// import java.util.HashSet;
import java.util.LinkedHashSet;
public class MyHashSet {
    public static void main(String[] args) {
        // HashSet h = new HashSet();
        LinkedHashSet h = new LinkedHashSet();
        System.out.println("Linked Hash Set: ");
        h.add("B");
        h.add("C");
        h.add("D");
        h.add("Z");
        h.add(10);
        h.add("M");
        h.add("E");
        System.out.println(h.add("Z"));
        System.out.println(h);
    }
}
```

Output:

```
Linked Hash Set:
false
[B, C, D, Z, 10, M, E]
```

In general we can use LinkedHashSet to develop cache based application, where duplicates are not allowed and insertion order is preserved.



SortedSet Interface

SortedSet interface is a child interface of **Set** interface. If we want to represent a group of individual objects, according to some sorting order, without duplicates, then we should go for **SortedSet** interface.

1. first() ⇒ 100
2. last() ⇒ 120
3. headSet(106) ⇒ [100, 101, 104]
4. tailSet(106) ⇒ [110, 115, 120]
5. subset(101, 115) ⇒ [101, 104, 106, 110]
6. comparator() ⇒ internal sorting technique

100
101
104
106
110
115
120

A = { 1, 2, 3 }

or

B = { 3, 2, 1 }

public interface SortedSet<E>
extends Set<E>

A Set that further provides a *total ordering* on its elements. The elements are ordered using their natural ordering, or by a Comparator typically provided at sorted set creation time. The set's iterator will traverse the set in ascending element order. Several additional operations are provided to take advantage of the ordering. (This interface is the set analogue of SortedMap.)

All elements inserted into a sorted set must implement the Comparable interface (or be accepted by the specified comparator). Furthermore, all such elements must be *mutually comparable*: e1.compareTo(e2) (or comparator.compare(e1, e2)) must not throw a ClassCastException for any elements e1 and e2 in the sorted set. Attempts to violate this restriction will cause the offending method or constructor invocation to throw a ClassCastException.

Note that the ordering maintained by a sorted set (whether or not an explicit comparator is provided) must be *consistent with equals* if the sorted set is to correctly implement the Set interface. (See the Comparable interface or Comparator interface for a precise definition of *consistent with equals*.) This is so because the Set interface is defined in terms of the equals operation, but a sorted set performs all element comparisons using its compareTo (or compare) method, so two elements that are deemed equal by this method are, from the standpoint of the sorted set, equal. The behavior of a sorted set *is* well-defined even if its ordering is inconsistent with equals; it just fails to obey the general contract of the Set interface.

All general-purpose sorted set implementation classes should provide four "standard" constructors: 1) A void (no arguments) constructor, which creates an empty sorted set sorted according to the natural ordering of its elements. 2) A constructor with a single argument of type Comparator, which creates an empty sorted set sorted according to the specified comparator. 3) A constructor with a single argument of type Collection, which creates a new sorted set with the same elements as its argument, sorted according to the natural ordering of the elements. 4) A constructor with a single argument of type SortedSet, which creates a new sorted set with the same elements and the same ordering as the input sorted set. There is no way to enforce this recommendation, as interfaces cannot contain constructors.

Note: several methods return subsets with restricted ranges. Such ranges are *half-open*, that is, they include their low endpoint but not their high endpoint (where applicable). If you need a *closed range* (which includes both endpoints), and the element type allows for calculation of the successor of a given value, merely request the subrange from lowEndpoint to successor(highEndpoint). For example, suppose that s is a sorted set of strings. The following idiom obtains a view containing all of the strings in s from low to high, inclusive:

```
SortedSet<String> sub = s.subSet(low, high+"\0");
```

A similar technique can be used to generate an *open range* (which contains neither endpoint). The following idiom obtains a view containing all of the Strings in s from low to high, exclusive:

```
SortedSet<String> sub = s.subSet(low+"\0", high);
```

This interface is a member of the Java Collections Framework.

Since:
1.2

Method Summary

All Methods | Instance Methods | Abstract Methods | Default Methods

Modifier and Type	Method and Description
`Comparator<? super E>`	`comparator()` Returns the comparator used to order the elements in this set, or `null` if this set uses the **natural ordering** of its elements.
`E`	`first()` Returns the first (lowest) element currently in this set.
`SortedSet<E>`	`headSet(E toElement)` Returns a view of the portion of this set whose elements are strictly less than `toElement`.
`E`	`last()` Returns the last (highest) element currently in this set.
`default Spliterator<E>`	`spliterator()` Creates a `Spliterator` over the elements in this sorted set.
`SortedSet<E>`	`subSet(E fromElement, E toElement)` Returns a view of the portion of this set whose elements range from `fromElement`, inclusive, to `toElement`, exclusive.
`SortedSet<E>`	`tailSet(E fromElement)` Returns a view of the portion of this set whose elements are greater than or equal to `fromElement`.

Methods inherited from interface java.util.Set

`add, addAll, clear, contains, containsAll, equals, hashCode, isEmpty, iterator, remove, removeAll, retainAll, size, toArray, toArray`

Methods inherited from interface java.util.Collection

`parallelStream, removeIf, stream`

Comparator.comparator() ← returns comparator object describes underlying sorting technique. If we are using DNSO, then we will get null.

Note :- DNSO → Ascending order
for Numbers → Ascending order
String → Alphabetical order

Set(I) 1.2V
SortedSet(I) 1.2V
NavigableSet(I) 1.6V
TreeSet 1.2V

TreeSet

- The underlying data structure is balanced tree.
- Duplicate objects are not allowed.
- Insertion order is not preserved.
- Heterogeneous objects are not allowed otherwise,we will get runtime exception ClassCastException.
- 'null' insertion is possible only once.(till 1.6Version of Java 'null' is allowed after that 'null' is not allowed at all.
- TreeSet implements Serializable, Cloneable but not RandomAccess.
- All objects will be inserted based on some sorting order. It. may be default natural sorting order or customized sorting order.

Constructors

TreeSet t = new TreeSet(); => Creates empty reset object where? all the objects entered into default natural sorting order.

TreeSet t = new TreeSet(Comparator c); => Creates an empty tree set object where? the element will be inserted according to customized sorting order specified by the comparator object.

TreeSet t = new TreeSet(SortedSet s); => Sorted order contained by object 's'.

TreeSet t = new TreeSet(Collection c); => DNSO, this is interconversion constructor.

```
import java.util.TreeSet;
public class TreeSetDemo{
    public static void main(String[] args) {
        TreeSet t = new TreeSet();
        t.add("A");
        t.add("a");
        t.add("L");
        t.add("Z");
        t.add("B");
        System.out.println(t);
        t.add(10);
    }
}
```

Homogenous ✗

Comparable ✓

Output:

```
[A, B, L, Z, a]
Exception in thread "main" java.lang.ClassCastException: class java.lang.String cannot be cast to class java.la
ng.Integer (java.lang.String and java.lang.Integer are in module java.base of loader 'bootstrap')
        at java.base/java.lang.Integer.compareTo(Integer.java:72)
        at java.base/java.util.TreeMap.put(TreeMap.java:849)
        at java.base/java.util.TreeMap.put(TreeMap.java:569)
        at java.base/java.util.TreeSet.add(TreeSet.java:259)
        at TreeSetDemo.main(TreeSetDemo.java:12)
```

Interface

1. fields ← public static final.
2. methods ← public abstract

Preview

```
void main() {
    =
    =
}
```

3. static method can be defined; Body will be implemented inside interface. ← 1.8
4. private method can be defined; ← 1.9
5. default method. can be defined by using default keyword. ← 1.11